DA350 Final project
Multiclass classification of Dry Beans using Machine learning techniques

Team members:

Ghadah Abu Fadalah

Fatima Al hajjeh

Shaima Al husain

Course instructor:

Dr.Enas Khashashneh

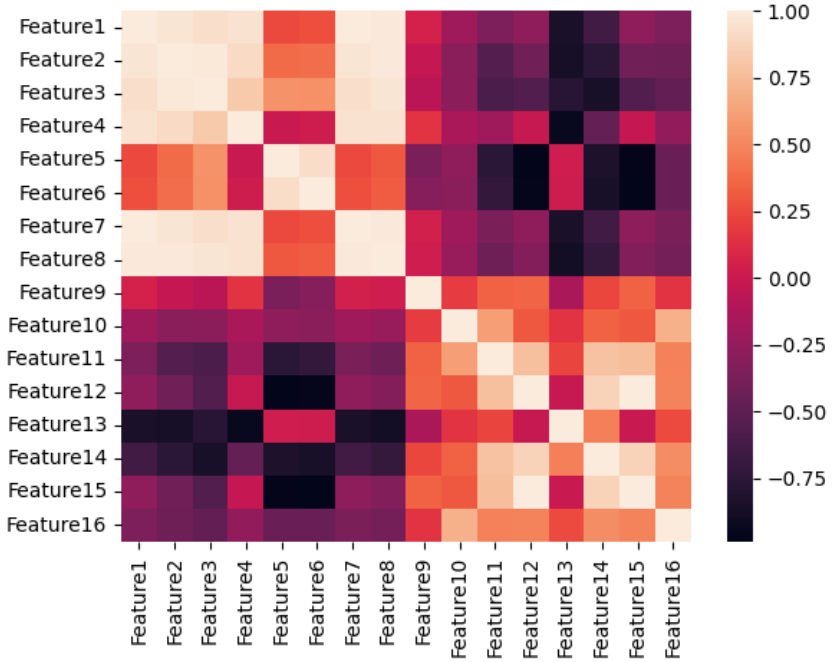
Problem Statement:

The problem addressed in this project is the classification of dry beans using machine learning techniques. The aim is to develop a model capable of accurately assigning each dry bean to one of seven distinct classes based on 16 different features. The challenge lies in creating a reliable and efficient machine learning model that can learn patterns from these features and make accurate predictions regarding the class to which each dry bean belongs.

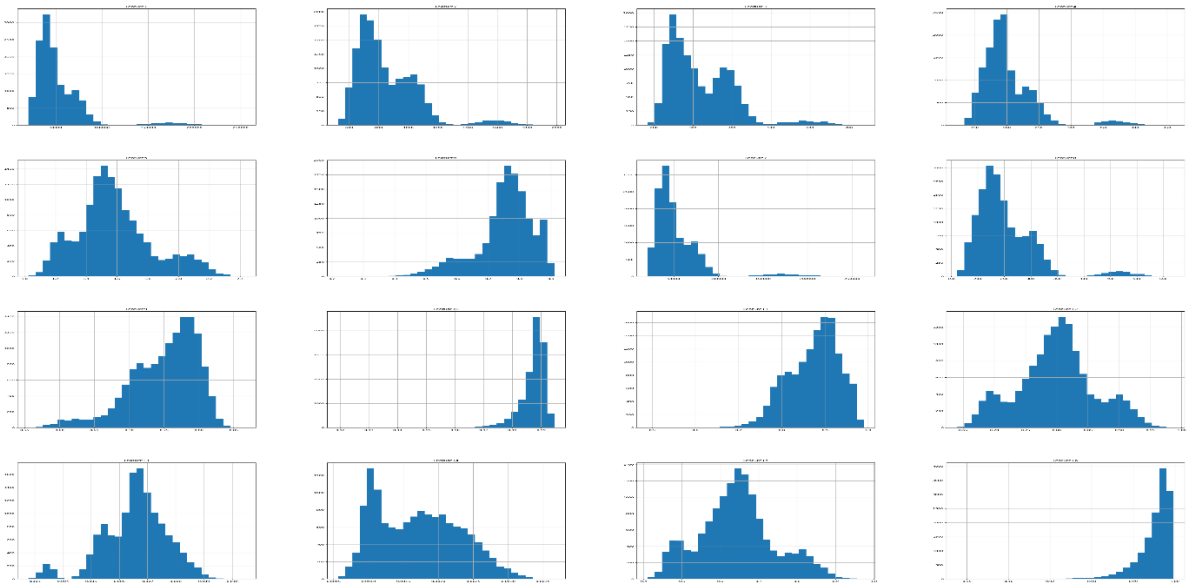
Dataset Description:

The dataset contains 13,612 samples, each representing a unique dry bean, with some characteristics. Before doing some preprocessing, we took a close look at the data in what we call Exploratory Data Analysis (EDA). Luckily, we didn't find any missing values.

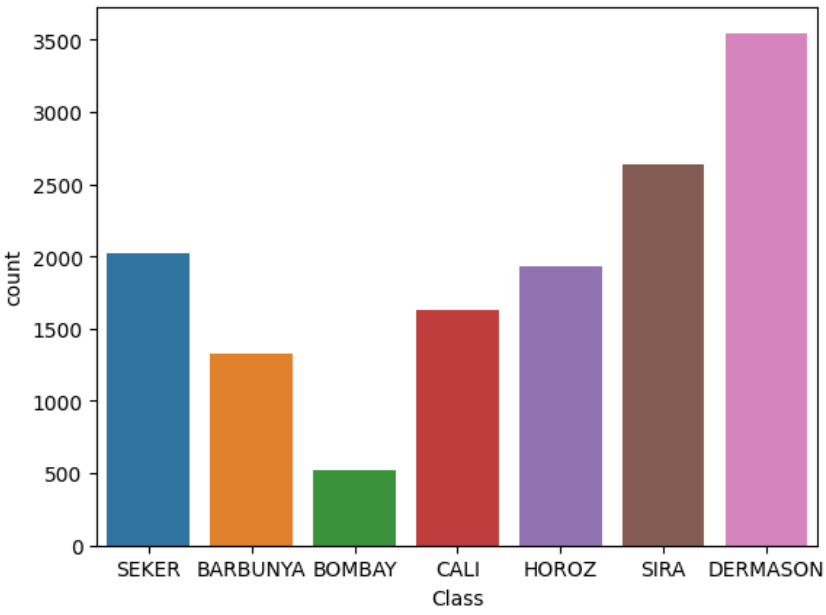
As we can see, some features in the datasets seemed to be highly correlated, and that could cause problems like multicollinearity and data sensitivity. To handle that, we used a dimensionality reduction technique called Principal Component Analysis (PCA) where we will discuss the process later, to simplify things without losing important information, and when we performed the PCA we reduced the features from 16 to 8.



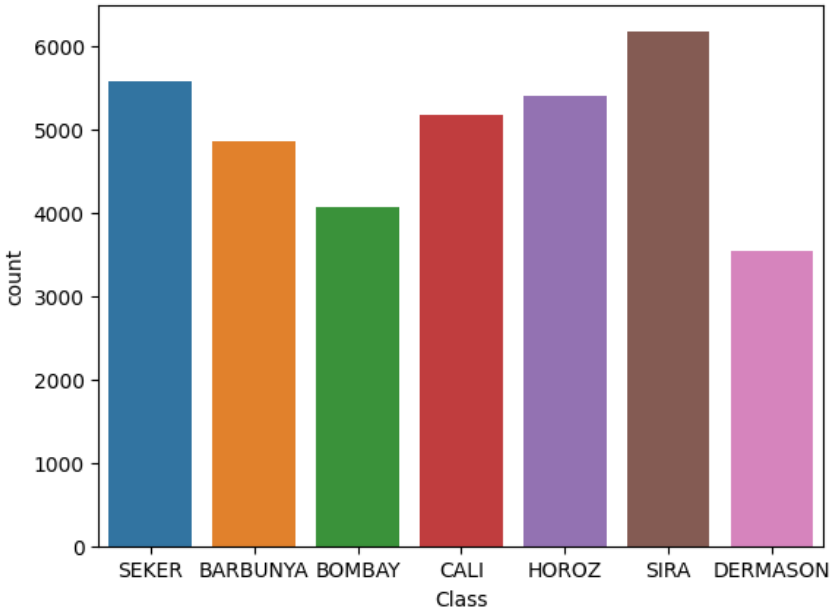
While checking how our features are distributed, we noticed that there are a lot of outliers, but removing them will introduce a new problem which is an imbalance within the dataset classes. And that's why we kept the outliers.



Lastly, we looked at the number of counts for each distinct class, and as we can see there's a bit of an imbalance. That's why we will perform up sampling in the minority classes.



After performing up sampling this is the result of the classes counts:



To address the issue of high correlation between features and enhance the efficiency of our model, we employed Principal Component Analysis (PCA). PCA is a dimensionality reduction technique that transforms the original features into a new set of uncorrelated variables called principal components. These components capture the maximum variance in the data. By using PCA, we aimed to reduce the dimensionality of our dataset while retaining the essential information. This process not only helps in dealing with multicollinearity but also aids in improving computational efficiency. And as mentioned before we reduced the features from 16 to the first 8 principal components.

In preparation for PCA, we standardized the data. This step is crucial for PCA because it ensures that all features contribute equally to the principal components, preventing the dominance of certain features due to differences in scale.

Lastly before splitting and building our models, we performed one-hot encoding. In machine learning, categorical data is often converted into a numerical format for models to better understand and learn from them. One-hot encoding transforms categorical variables into binary vectors, where each category becomes a separate column with binary values (0 or 1). Although, some ML models can work with categorical data, encoding is important because it prevents the model from misinterpreting categorical variables as ordinal or having an inherent order, ensuring accurate representation of the categorical information, and improving the model's performance in handling categorical classes. And this will lead to having multilabel target with 7 labels for each class of the 7 classes.

Experimental Design:

In this project, we explored the performance of three classifiers: K-Nearest Neighbors (KNN), Decision Tree, and Random Forest. Our dataset was divided into training and test sets using an 80-20 split, with 20% for testing and 80% for training. Additionally, a validation set, which was 10% of the training data.

EXPERIMENT 1 K-NEAREST NEIGHBOR:

KNN is a simple and effective classification algorithm that classifies an unknown data point based on the majority class of its k-nearest neighbors in the feature space. It makes predictions by considering the classes of nearby data points, where the distance metric determines the "closeness" of points.

KNN is particularly suitable when there is a natural clustering of similar instances in the feature space. In the context of classifying dry beans, where beans with similar characteristics might belong to the same class, KNN is well-suited to capture such patterns.

Training Phase:

The KNN model was initially trained using 70% of the training set (80% of the overall data) from the first split.

Validation Phase:

Model performance was evaluated on the validation set, constituting 10% of the training set from the first split.

Initial Performance:

Initial model accuracy: 92.89

Initial F1 score: 93.89

Hyperparameter Tuning:

A hyperparameter tuning process was conducted to optimize the model's parameters. We used Grid Search to find the best hyperparameter for our model which the number of nearest neighbors.

Updated Performance:

Final Accuracy: 95.52

Final F1 score: 95.61

EXPERIMENT 2 DECISION TREE:

A Decision Tree recursively splits the dataset into subsets based on features, creating a tree-like structure. It makes decisions by traversing the tree from the root to a leaf node. Each internal node represents a decision based on a feature, and each leaf node represents the predicted class.

Decision Trees are advantageous for tasks involving complex decision-making based on multiple features. In the case of classifying dry beans, Decision Trees can capture intricate relationships between bean features, aiding in the identification of class-specific patterns.

Training Phase:

The Decision Tree model was trained using the same training set split.

Validation Phase:

Model performance was assessed on the validation set.

Initial Performance:

Initial model accuracy: 93.83

Initial F1 score: 94.78

Hyperparameter Tuning:

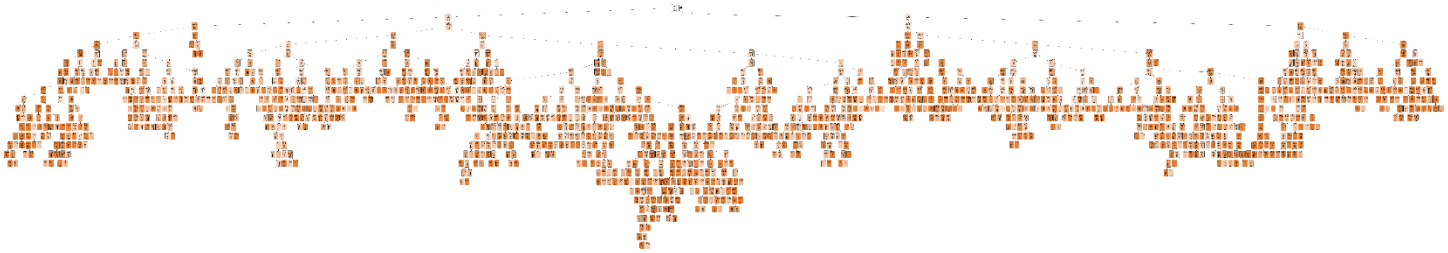
Like KNN, hyperparameter tuning process was conducted to optimize the model's parameters. We used Grid Search to find the best hyperparameter for our model and here the hyperparameters were the maximum depth, minimum samples split, and minimum samples leaf.

Updated Performance:

Final Accuracy: 95.59

Final F1 score: 96.15

Here is a visualization of our Decision Tree model:



WE WILL PROVIDE THE PICTURE IN THE PROJECT'S FOLDER FOR A BETTER QUALITY

EXPERIMENT 3 RANDOM FOREST:

Random Forest is an ensemble learning method that constructs multiple Decision Trees during training and outputs the mode of the classes (classification) of the individual trees. It introduces randomness by training each tree on a random subset of the data and using a random subset of features for each split.

Random Forests are powerful and robust, often preventing overfitting associated with a single Decision Tree. In scenarios where the data may exhibit complex interactions, Random Forests can capture diverse patterns, making them suitable for classifying dry beans with multiple features and potential non-linear relationships.

Training Phase:

The Random Forest model was trained using the same training set split.

Validation Phase:

Model performance was assessed on the validation set.

Initial Performance:

Initial model accuracy: 92.46

Initial F1 score: 94.37

Hyperparameter Tuning:

Similar to KNN and Decision Tree, hyperparameter tuning process was conducted to optimize the model's parameters. We used Grid Search to find the best hyperparameter for our model and here the hyperparameters were similar to the Decision Tree's hyperparameters which is the maximum depth, minimum samples split, and minimum samples leaf.

Updated Performance:

Final Accuracy: 97.80

Final F1 score: 97.97

Here is a visualization of our Random Forest model:



WE WILL PROVIDE THE PICTURE IN THE PROJECT'S FOLDER FOR A BETTER QUALITY

Conclusion

After testing different Machine Learning models to classify dry beans, we found that the Random Forest model worked the best. It was better at understanding the patterns in the features of the beans compared to the other models like KNN and Decision Trees.
The Random Forest model, functioning as an ensemble method of decision trees, demonstrated superior performance in navigating the complexities of our dataset. Its collective decision-making approach allowed it to effectively handle intricate patterns within the diverse features of dry beans. The model's ability to mitigate overfitting and capture diverse patterns positions it as the optimal choice for precise and reliable classification of various dry bean types. The ensemble nature of Random Forest proved instrumental in achieving robust performance, making it a technically sound solution for our classification task.

Here is a visualization of the models' comparison:

